# Тема 7. Теоретическое задание

1. Рассмотрите `error` и `try-with` как *производную форму*, реализующую обработку ошибок через типы-суммы [1, § 11.9].

   - `error` ведёт себя как `inl unit`;
   - константы (например, `0`) оборачиваются в `inr` (например, `inr 0`);
   - все прочие конструкции явно обрабатывают тип `Unit + T` вместо выражения типа `T`; например, для применения $e(\mathtt{t}_1\ \mathtt{t}_2)$:

   ```
   case t1 of
     inl u => inl u
   | inr t1' =>
       case t2 of
         inl u => inl u
       | inr t2' => inr (e(t1') e(t2'))
   ```

   (a) Выпишите явно функцию раскрытия сокращений для $\lambda$-исчисления с логическими и арифметическими выражениями [1, §§ 8–9] и со *всеми* простыми расширениями из [1, § 11]. Если для какого-то расширения **невозможно** раскрытие, сохраняющее семантику исходных правил вычисления для `error` и `try-with` [1, § 14.1], то кратко объясните, почему.

   (b) Покажите, что раскрытие сохраняет семантику исходных правил вычисления для `error` и `try-with` [1, § 14.1]. Для доказательства достаточно рассмотреть только простое типизированное $\lambda$-исчисление с логическими [1, §§ 8–9], без арифметических выражений и дополнительных расширений.

2. Рассмотрите расширение простого типизированного $\lambda$-исчисления с обработкой ошибок со значениями [1, § 14.3] конструкцией

   $$\texttt{with exception}\ \mathcal{E}\ \texttt{do}\ t_1\ \texttt{catch}\ x \Rightarrow t_2.$$

   Эта конструкция:

   - вводит тип исключений $\mathtt{T}_{\mathsf{exn}} = \mathcal{E}$ *временно*, только на время вычисления $t_1$;
   - при возникновении ошибки (`raise`) управление передаётся ветке `catch` $x \Rightarrow t_2$, где $x$ — переменная, с которой связывается значение ошибки типа $\mathcal{E}$;
   - в $t_2$ допускается использование переменной $x$.

   (a) Выпишите явно правила вычисления для этой конструкции.

   (b) Предложите правила типизации для этой конструкции, а также новые (уточнённые) правила типизации `raise` и `try-catch`, принимающие во внимание локальный тип исключений. При необходимости расширьте отношение типизации (например, добавьте текущий тип исключений в контекст или введите суждение вида $\Gamma \vdash_{\mathcal{E}} t : T$).

   (c) Аргументируйте, почему предложенные правила сохраняют безопасность типизации, или укажите причину, по которой предложенная конструкция не может быть безопасной для системы типов. Подробное доказательство безопасности типизации не требуется.

## Список литературы


[1] Б. Пирс. *Типы в языках программирования: пер. с англ.* Лямбда пресс, 2012. ISBN: 9785791300829. URL: https://books.google.ru/books?id=HJJCkgEACAAJ.